

PENTEST REPORT

KeuzeWijzer Pentest Report

Informatica

Group: A2

Students:

- Christian Horler
- Angel Zeng
- Martijn van Houwelingen
- Wassim Balouda
- Jeftha van Twillert

Date: January 18, 2026

Version: 1.3

Table of Contents

List of Figures	3
1 Executive Summary	4
1.1 Overview	4
1.2 Identified Vulnerabilities	4
2 Methodology	6
2.1 Approach	6
2.2 Tools	7
2.3 Limitations	11
2.4 Scope	12
3 OWASP Top 10	13
4 Findings	14
I 1: SSH Fail2Ban Policy	14
L 1: MariaDB Publieke Toegankelijkheid	16
M 1: Non-Functional Rate Limiting	18
H 1: Interne applicatielogs worden gedeeld via Discord	22
C 1: Exposed Sensitive Authentication Credentials	25
5 Document History	28

List of Figures

Figure 1 - Distribution of identified vulnerabilities 5

1 Executive Summary

1.1 Overview

In het kader van dit schoolproject is een beveiligingsassessment uitgevoerd op **Module Keuze Wijzer**, een webapplicatie met een **Svelte (Vite) frontend** en een **NestJS backend API**. Het doel van deze pentest was het identificeren van kwetsbaarheden en zwakke plekken die kunnen leiden tot ongeautoriseerde toegang, datalekken, misbruik van functionaliteit of verstoring van de betrouwbaarheid van de applicatie. De test is uitgevoerd met focus op de **OWASP Top 10:2025 (A01-A10)**, met nadruk op o.a. **Broken Access Control, Identification & Authentication Failures, Security Misconfiguration, Cryptographic Failures**, en **Mishandling of Exceptional Conditions**.

De assessment is uitgevoerd binnen de afgesproken scope en met minimaal invasieve technieken. Er is getest met meerdere identiteiten om scheiding tussen rollen en object-level toegang te beoordelen: **anoniem, Student A, Student B** en **Teacher**. Zowel de gebruikersinterface als directe API-aanroepen zijn beoordeeld. Specifieke aandacht is besteed aan de implementatie van **JWT access tokens** en **refresh tokens**, inclusief sessiegedrag bij token-expiratie en logout, en het testen van misbruikscenario's zoals brute-force en tokenmisbruik.

De resultaten laten zien dat er een aantal **serieuze verbeterpunten** zijn, vooral rondom **secrets/logging** en **exposure van kritieke diensten**. De grootste risico's kwamen niet alleen uit "klassieke" webbugs, maar uit de manier waarop credentials en tokens worden opgeslagen en gedeeld. In Discord zijn **volledige credentials** aangetroffen voor **SSH, Portainer en MariaDB** (inclusief root- en applicatieaccounts). Daarnaast zijn in Discord-logs ook **refresh tokens** en **user IDs** zichtbaar. Dit creëert een direct pad naar **account takeover** en mogelijk zelfs volledige servercompromittering wanneer iemand toegang krijgt tot dat kanaal of wanneer een Discord-/developer account wordt overgenomen. Dit raakt met name aan **A04 (Cryptographic Failures)** en **A02 (Security Misconfiguration)**.

Daarnaast is vastgesteld dat **rate limiting/backoff ontbreekt op login en registratie**, waardoor brute-force/credential stuffing en misbruik van accountcreatie mogelijk is (**A07/A02**). Ook blijkt dat **MariaDB publiek bereikbaar is op poort 3306** en dat **TLS niet wordt afgedwongen**: een verbinding met `--ssl=OFF` werkt en `Ssl_version` blijft leeg. Dit vergroot de attack surface en maakt onversleutelde databaseverbindingen mogelijk (**A02/A04**).

Prioriteit voor opvolging ligt bij het direct verwijderen/roteren van gelekte secrets en het stoppen van token- en credential logging, gevolgd door het beperken van publiek bereikbare beheer- en databasepoorten en het afdwingen van TLS, en het implementeren van rate limiting op de authenticatie-endpoints.

1.2 Identified Vulnerabilities

#	Severity	Description	Page
I 1	Info	SSH Fail2Ban Policy	14
L 1	Low	MariaDB Publieke Toegankelijkheid	16
M 1	Medium	Non-Functional Rate Limiting	18

#	Severity	Description	Page
H 1	High	Interne applicatielogs worden gedeeld via Discord	22
C 1	Critical	Exposed Sensitive Authentication Credentials	25

Vulnerability Overview

In the course of this penetration test **1 Critical**, **1 High**, **1 Medium**, **1 Low** and **1 Info** findings were identified:

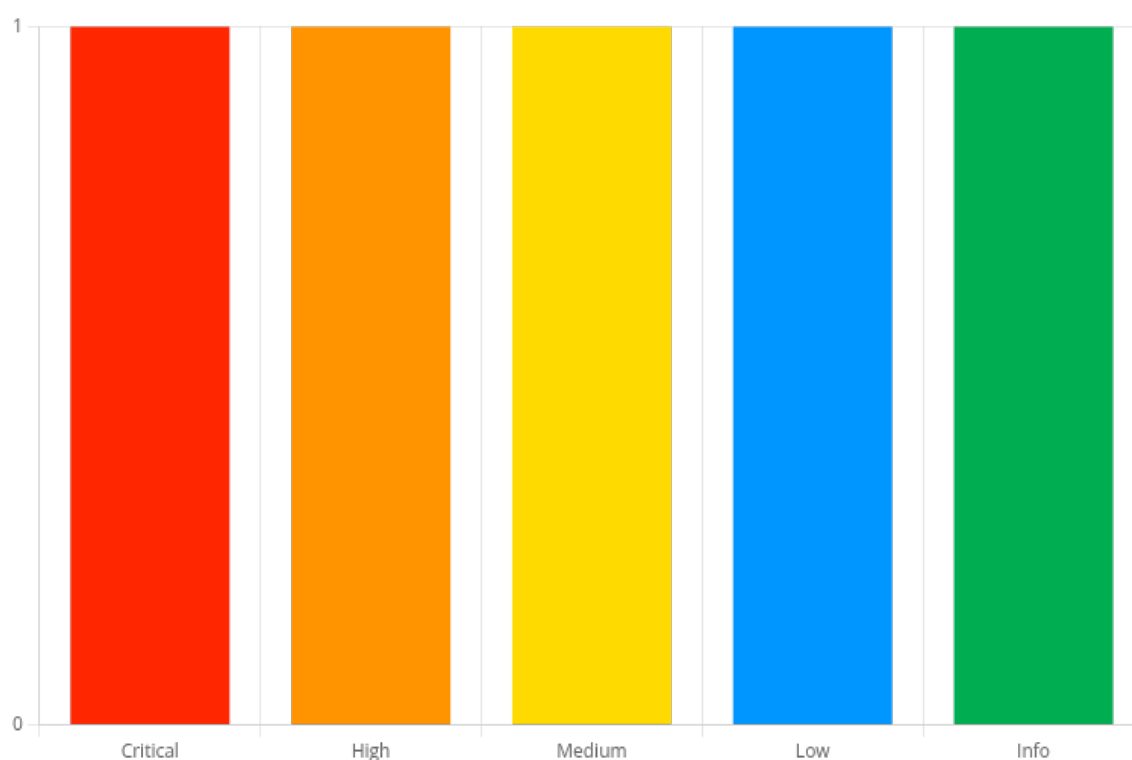


Figure 1 - Distribution of identified vulnerabilities

2 Methodology

2.1 Approach

Deze pentest is uitgevoerd met het doel om **realistische aanvalspaden** te vinden die een echte aanvaller ook zou gebruiken: van “wat staat er open?” tot “kan ik een account overnemen, rechten verhogen of data uitlezen?”. De OWASP Top 10:2025 is gebruikt als **classificatie en checklist** om te borgen dat de meest voorkomende webapp-risico's worden afgedekt, maar de test zelf is vooral gestuurd door een praktische denkwijze: **aanvalsoppervlak** → **hypotheses** → **testen** → **bewijs** → **impact** → **fix**.

- We zoeken eerst uit **wat er bestaat** (endpoints, rollen, services, integraties).
- Daarna bepalen we **waar het mis kan gaan** (auth flows, autorisatie, tokens, logging, configuratie).
- Vervolgens testen we gericht met **low-impact** stappen, zodat bevindingen reproduceerbaar zijn zonder onnodige verstoring.

De aanpak volgt een “breed eerst, dan dieper” structuur:

1. Recon & Attack Surface Mapping

Inventariseren wat er bereikbaar is (web, API, admin tooling, databasepoorten), welke technologieën gebruikt worden, en hoe de applicatie is opgebouwd. Hierbij wordt gekeken naar open poorten/services, domeinen/subdomeinen, headers, en het API-verkeer dat de frontend genereert.

2. Context & threat-driven hypotheses

Op basis van de gevonden componenten formuleren we test-hypotheses. Voorbeeld:

- “Als er student/teacher rollen zijn, kan een student dan data van een andere student zien?” (Broken Access Control)
- “Als er refresh tokens zijn, worden die correct beschermd en geroteerd?” (Auth failures / crypto)
- “Als er logs of error handling bestaan, lekt dat interne details of secrets?” (Misconfig/logging/exception handling)

3. Gericht testen van kernflows

Er is extra aandacht besteed aan flows die bijna altijd high impact geven als ze fout zijn:

- **Authenticatie:** login, registratie, refresh tokens, logout, sessiegedrag
- **Autorisatie:** rolchecks, IDOR/BOLA, horizontale/verticale privilege escalation
- **Input & API:** manipulatie van parameters/IDs, foutafhandeling, injection-pogingen
- **Server/infra:** exposed services, TLS, admin interfaces, secrets/credentials buiten de juiste plek

4. Bewijs verzamelen en impact afbakenen

Elke bevinding wordt vastgelegd met reproduceerbare stappen, request/response bewijs, timestamps, en een duidelijke beschrijving van wat een aanvaller hiermee kan. Waar mogelijk is gekozen voor **minimale impact**: aantonen dat iets kan, zonder maximaal misbruik.

5. Classificatie en rapportage volgens OWASP Top 10:2025

Bevindingen zijn gemapt naar de OWASP Top 10:2025 categorieën om consistentie te houden in risico-uitleg en prioritering. Dit helpt ook om aan te tonen dat er breed getest is (niet alleen “een paar random checks”)

-
- **Assume breach mindset:** we testen alsof een aanvaller al een normaal account heeft (student) en kijkt hoe ver die kan komen.
 - **Least effort for maximum impact:** eerst de meest waarschijnlijke fouten (access control, auth, misconfig, secrets/logging).
 - **Low-impact bewijsvoering:** geen DoS/stress tests, geen destructieve datawijzigingen; liever aantonen met beperkte requests en testaccounts.
 - **Verifiëren i.p.v. aannemen:** “lijkt geconfigureerd” telt niet; alleen wat aantoonbaar afdwingbaar is (bv. rate limiting dat echt 429 geeft, TLS dat verplicht is, etc.).

Koppeling aan OWASP Top 10:2025

De volgende thema's vormen de test, omdat ze het vaakst tot echte impact leiden:

- **Broken Access Control (A01):** kan een gebruiker bij andermans data of admin acties?
- **Security Misconfiguration (A02):** onnodig open services, debug/info exposure, ontbrekende limits
- **Cryptographic Failures (A04):** token/credential exposure, TLS/transport, veilige omgang met secrets
- **Injection (A05):** inputmanipulatie in parameters/body/headers
- **Authentication Failures (A07):** brute force bescherming, token lifecycle, session management
- **Logging/Monitoring en exception handling (A09/A10):** lekken logs stack traces, secrets in logs

Deze categorieën zijn gebruikt om testcases te structureren en om bevindingen helder te positioneren in het rapport.

2.2 Tools

Voor deze pentest is een mix gebruikt van **netwerk-/service discovery**, **web & API testing**, en **server-side verificatie**. De tools zijn vooral ingezet om (1) het aanvalsoppervlak in kaart te brengen, (2) hypothesen snel te kunnen valideren met reproduceerbare requests, en (3) bevindingen met hard bewijs te onderbouwen (logs, statuscodes, headers, configuratie-output).

Netwerk & service discovery

Nmap

- **Doel:** identificeren welke services extern bereikbaar zijn, op welke poorten, en (waar mogelijk) welke softwareversies draaien.
- **Gebruik in de test:**
 - Service/versiedetectie om “exposure” te onderbouwen (bijv. open DB-poort, admin interfaces).
 - Baseline scan als startpunt voor verdere verdiepende tests.
- **Voorbeeld (typische aanpak):**
 - `nmap -sV -p- <IP>` (poortinventarisatie + versie-indicatie)
 - eventueel beperkt: `nmap -sV -sC -p 80,443,3306,9000 <IP>` (alleen waar passend en binnen scope)
- **Output als bewijs:** Nmap output is opgeslagen en gebruikt in de evidence-sectie van relevante bevindingen (bijv. open poorten en service fingerprints).
- **Beperkingen:** versieherkenning is soms een best-guess; resultaten zijn altijd handmatig geverifieerd met server-side checks of aanvullende tooling.

HTTP/TLS controles (transport & headers)

curl

- **Doel:** snel verifiëren van HTTP-responses, headers en redirects zonder browser.
- **Gebruik in de test:**
 - check van security headers (`-I`), redirects, statuscodes.
 - vergelijken van gedrag tussen endpoints/ports (bijv. reverse proxy vs direct exposed service).
- **Voorbeelden:**
 - `curl -I https://<domain>`
 - `curl -vk https://<domain>` (debug van TLS/handshake indien nodig)

OpenSSL (s_client)

- **Doel:** low-level inspectie van TLS handshake/certificaatketen (servercert, protocol, ALPN).
- **Gebruik in de test:**
 - bevestigen van certificaatdetails en welke protocollen onderhandeld worden.
- **Voorbeeld:**
 - `openssl s_client -connect <domain>:443 -servername <domain>`

(Optioneel) testssl.sh

- **Doel:** bredere TLS-beoordeling (protocols/ciphers/misconfig checks) op HTTPS endpoints.
- **Gebruik in de test:** alleen waar relevant en zonder agressieve load.
- **Beperkingen:** tooling geeft veel output; bevindingen zijn pas opgenomen wanneer ze reproduceerbaar en relevant waren.

Webapplicatie & API testing

Burp Suite

- **Doel:** het centraal platform voor web/API pentesting: onderscheppen, herhalen, manipuleren en vergelijken van requests/responses.
- **Gebruik in de test:**
 - **Proxy/Intercept:** het verkeer van de frontend (Svelte/Vite) naar de backend (NestJS) analyseren.
 - **Repeater:** requests herhalen met aangepaste IDs/parameters (IDOR/BOLA), headers, body, rollen/tokens.
 - **Comparer:** verschillen analyseren in responses (bijv. user enumeration, autorisatieverschillen).
 - **Intruder (beperkt):** alleen "low-impact" (bijv. kleine bursts voor rate limiting verificatie), geen hoge load.
- **Token handling:** JWT/refresh tokens zijn per testaccount gescheiden gehouden (aparte sessions/contexts) om rolchecks betrouwbaar te testen.
- **Evidence:** relevante requests/responses zijn geëxporteerd of gescreenshot (met redactie van secrets).

Postman / Insomnia

- **Doel:** gestructureerd testen van API endpoints buiten de browser om, met herbruikbare collections en environment variabelen.
- **Gebruik in de test:**
 - vaste set API-calls (login/refresh/logout, CRUD endpoints)
 - snelle reproduceerbaarheid voor bewijs en retests
 - expliciet kunnen sturen van headers/body zonder browser-side gedrag
- **Evidence:** collections/requests zijn gebruikt om reproduceerstappen consistent vast te leggen.

Browser DevTools (Network tab)

- **Doel:** observatie van de “echte” applicatieflow: welke endpoints worden aangeroepen, met welke payload, en wat is de UI-trigger.
 - **Gebruik in de test:**
 - baseline mapping van API routes
 - identificeren van object IDs, query parameters en rolgebonden calls
-

Fuzzing & endpoint discovery (beperkt en gecontroleerd)

ffuf

- **Doel:** gecontroleerde discovery van verborgen/ongebruikte endpoints of paden (waar toegestaan).
 - **Gebruik in de test:**
 - alleen binnen scope en met rate limiting om geen load/DoS te veroorzaken
 - vooral op bekende prefixen (bijv. `/api/`, `/admin`, `/health`, `/swagger`)
 - **Beperkingen:** directory fuzzing kan veel requests genereren; daarom zijn tijdvensters, threads en rate limits beperkt gehouden.
-

Server-side verificatie (host en containers)

ss / lsof

- **Doel:** “ground truth” vaststellen: welke processen luisteren echt op welke interfaces/poorten.
- **Gebruik in de test:**
 - verifiëren of services op `0.0.0.0` / `[::]` luisteren (extern bereikbaar) vs alleen localhost/intern
- **Voorbeelden:**
 - `ss -tulpn`
 - `lsof -i -P -n`

systemctl / journalctl

- **Doel:** status en logs van services bekijken (bijv. fail2ban, docker, webserver).
- **Gebruik in de test:**
 - bevestigen dat security controls actief zijn
 - log-based bewijs voor bans/errors (zonder gevoelige data te kopiëren)

Docker tooling

- **Doel:** inzicht in container exposure (ports), proxies en runtime configuratie.
- **Gebruik in de test:**
 - identificeren van `docker-proxy` gebonden poorten en public exposure
 - verifiëren welke containers/services verantwoordelijk zijn voor open poorten

Fail2Ban tooling

- **Doel:** verifiëren of brute-force mitigaties werken (bijv. op SSH).
- **Gebruik in de test:**
 - `fail2ban-client status` en `fail2ban-client status sshd` om bans en counters te controleren

MariaDB client

- **Doel:** verifiëren van transport security en servergedrag bij (on)versleutelde DB-verbindingen.
 - **Gebruik in de test:**
 - controleren of TLS sessies gebruikt worden (`SHOW STATUS LIKE 'Ssl_version'`)
 - aantonen dat TLS **niet afgedwongen** is door verbinding zonder TLS te forceren (`--ssl=OFF`) en `Ssl_version` leeg te zien
-

Bewijsverzameling & redactieregels

Bewijs (artefacts)

- Nmap outputs, command outputs (ss/fail2ban/mariadb), screenshots, en request/response captures.
- Waar mogelijk zijn timestamps vastgelegd om reproduceerbaarheid te vergroten.

Redactie

- Tokens, wachtwoorden, refresh tokens, cookies, en andere secrets zijn **gemaskeerd** in het rapport (bijv. alleen eerste/laatste tekens zichtbaar).
 - Bij gevoelige bevindingen is bewijs geleverd zonder “misbruik” (bijv. aantonen dat credentials zichtbaar zijn, zonder daadwerkelijk in te loggen).
-

Toolbeperkingen en kwaliteitscontrole

- Tools leveren soms **false positives** of onvolledige fingerprinting; daarom zijn bevindingen altijd **handmatig bevestigd** (bijv. server-side `ss` output naast Nmap).
- Scans en fuzzing zijn **bewust beperkt** uitgevoerd om beschikbaarheid niet te beïnvloeden.
- Waar tooling iets “mogelijk” suggereerde, is het pas als finding opgenomen wanneer het **reproduceerbaar** en **impactvol** was.

2.3 Limitations

Scope en autorisatie

- De test is uitgevoerd **alleen binnen de afgesproken scope** (doelhost(s), domeinen en applicatieonderdelen die expliciet zijn toegestaan).
- Systemen of third-party diensten buiten scope (bijv. externe SaaS/integraties) zijn niet actief aangevallen, behalve waar ze direct onderdeel zijn van de applicatieflow en alleen op observatieniveau.

Geen versturende testen (beschikbaarheid)

- Er is **geen DoS/stress/load testing** uitgevoerd. Dit betekent dat de pentest geen uitspraken doet over performance onder aanvalslast of DDoS-weerbaarheid.
- Rate limiting/backoff testen zijn bewust **low-impact** uitgevoerd (kleine aantallen requests) om lockouts of verstoring te voorkomen.

Beperkte exploitatie

- Bevindingen zijn aangetoond met **minimale noodzakelijke stappen**. Er is niet geprobeerd om "maximaal misbruik" uit te voeren (bijv. volledige account takeover) wanneer de impact al overtuigend aangetoond kon worden.
- Bij gevoelige bevindingen (credentials/tokens) is het bewijs geleverd door het **aantonen van exposure**, niet door het daadwerkelijk gebruiken van die secrets om in te loggen.

Tijd en testdekking

- Door tijds- en scopebeperkingen is niet ieder endpoint en iedere edge case volledig uitputtend getest.
- De focus lag op de meest kritieke aanvalsoppervlakken:
 - authenticatie (login/registratie/refresh)
 - autorisatie (rollen/IDOR/BOLA)
 - misconfiguraties (exposed services, TLS, logging/secrets)
 - error handling / informatielekken
- "Negative testing" (alle mogelijke foutscenario's) is uitgevoerd waar relevant, maar niet in alle mogelijke varianten.

Toegangsniveau en afhankelijkheden

- Sommige controles zijn afhankelijk van beschikbaar testmateriaal:
 - testaccounts met verschillende rollen (student/teacher)
 - toegang tot server-side commando's/logs voor verificatie (waar beschikbaar)
- Resultaten kunnen beïnvloed worden door omgevingsfactoren zoals:
 - caching/CDN of reverse proxy gedrag
 - rate limits op infrastructuurniveau die verschillen van applicatie-level controles
 - configuratieverschillen tussen development/staging/production

Behandeling van bewijs en privacy

- In het rapport zijn gevoelige waarden (wachtwoorden, tokens, cookies, secrets) **altijd gemaskeerd**.
- Waar PII in scope viel (bijv. user IDs/e-mails), is alleen opgenomen wat nodig was om impact aan te tonen, met minimale blootstelling.

Aannames

- De test gaat ervan uit dat de geteste omgeving representatief is voor de beoogde productieomgeving (configuratie, rollen, endpoints).
- Waar dit niet zeker is, is dit expliciet benoemd bij de betreffende bevinding (bijv. "afhankelijk van productieconfiguratie").

2.4 Scope

Testing was performed from **Jan 14, 2026** to **Jan 16, 2026** (Christian Horler).

In-scope

- Server: 157.173.124.159
- Domain: https://*.157.173.124.159
- Discord Channel
- Any relevant project information.

Target:

3 OWASP Top 10

Version: OWASP 2025

OWASP Category	Tested	Result	Notes
A01 Broken Access Control	✓	OK	No findings
A02 Security Misconfiguration	✓	Open	2 Findings
A03 Software Supply Chain Failures	✓	OK	No findings
A04 Cryptographic Failures	✓	Open	1 Finding
A05 Injection	✓	OK	No findings
A06 Insecure Design	✓	Open	2 Findings
A07 Authentication Failures	✓	OK	No findings
A08 Software or Data Integrity Failures	✓	OK	No findings
A09 Security Logging and Alerting Failures	✓	OK	No findings
A10 Mishandling of Exceptional Conditions	✓	OK	No findings

4 Findings

I 1: SSH Fail2Ban Policy	
Severity	Info
OWASP	A02:2025 - Security Misconfiguration
Affected components	SSH <user>@<ServerIP>
References	https://www.hostinger.com/tutorials/fail2ban-configuration#h-how-to-set-up-fail2ban

Overview

Op de server is **Fail2Ban** actief voor **SSH** en dit is tijdens de test ook aantoonbaar afgedwongen. Na meerdere mislukte SSH-inlogpogingen binnen een korte periode werd het bron-IP tijdelijk geblokkeerd. Dit is geen kwetsbaarheid, maar een **informatieve bevinding** die laat zien dat er een effectieve basismaatregel tegen brute-force/credential-stuffing op SSH aanwezig is.

Momenteel zouden de **Fail2Ban** regels wel wat strenger kunnen zijn om bots beter te vermijden.

Details

Fail2Ban monitort SSH-authenticatiepogingen via de `sshd` jail en leest hiervoor de (zoals gedefinieerd via `logpath = %(sshd_log)s` in the `jail.conf`). Bij herhaalde mislukte inlogpogingen binnen het ingestelde tijdsvenster wordt het bron-IP automatisch geblokkeerd door een firewall-actie.

De relevante drempels in de configuratie zijn:

- `maxretry = 5` (aantal mislukte pogingen)
- `findtime = 10m` (tijdsvenster waarin die pogingen tellen)
- `bantime = 10m` (duur van de blokkade)

Hiermee wordt brute-force gedrag automatisch gedetecteerd en tijdelijk afgesneden.

Impact

Positieve security impact (hardening)

- Vermindert de kans op succesvolle brute-force/credential-stuffing aanvallen op SSH.
- Beperkt de belasting door geautomatiseerde scan- en inlogpogingen.
- Verhoogt de drempel voor opportunistische aanvallers en bots.

Beperking: dit beschermt specifiek **SSH**, en vervangt geen goede basisconfiguratie (zoals key-based auth, beperkte exposure, allowlisting/VPN).

Reproduction

1. Maak verbinding naar de server via SSH vanaf één bron-IP.
2. Voer binnen **10 minuten** in totaal **5 mislukte** inlogpogingen uit (bijv. verkeerde credentials).
3. Controleer de status van de jail:
 - `fail2ban-client status sshd`
4. Observeer dat het bron-IP in de **Banned IP list** verschijnt.
5. Probeer opnieuw te verbinden via SSH vanaf hetzelfde IP en observeer dat de verbinding wordt geweigerd/geblokkeerd gedurende de ingestelde **bantime**.

Evidence

```
ssh user@<SERVER_IP>

user@<SERVER_IP>'s password:
Permission denied, please try again.
user@<SERVER_IP>'s password:
Permission denied, please try again.
user@<SERVER_IP>'s password:
Permission denied, please try again.
user@<SERVER_IP>'s password:
Permission denied, please try again.
user@<SERVER_IP>'s password:
Permission denied, please try again.
user@<SERVER_IP>'s password:
Permission denied, please try again.
```

Hierna als je binnen 10 minuten weer probeert in te loggen

```
ssh user@<SERVER_IP>
ssh: connect to host <SERVER_IP> port 22: Connection refused
```

Recommendation

Behouden: laat Fail2Ban voor SSH actief staan.

Hardening:

- Verhoog **bantime** of gebruik incrementele bantime voor herhaalde offenders.
- Beperk SSH-toegang waar mogelijk via **VPN/jump host** of **IP allowlisting**.
- Gebruik bij voorkeur **key-based authentication** en schakel wachtwoorden uit (`PasswordAuthentication no`).

Retest

Status: Accepted

L 1: MariaDB Publieke Toegankelijkheid

Severity	Low
OWASP	A06:2025 - Insecure Design
Affected components	157.173.124.159:3306 (MariaDB)
References	

Overview

De MariaDB database is direct vanaf het internet bereikbaar op **port 3306**. Dat is onnodig en maakt de server een stuk aantrekkelijker voor scanning en brute-force pogingen. Daarnaast is **TLS niet verplicht**, waardoor databaseverkeer (inclusief credentials en data) in plaintext kan verlopen zodra een client zonder TLS verbindt.

Details

Tijdens een simpele nmap scan werd **3306/tcp** als open gevonden en herkend als **MariaDB**. Met `ss -tulnp` is het ook te zien dat het luistert op **0.0.0.0** via `docker-proxy`. Praktisch betekent dit dat de database is "gepubliceerd" en dus niet alleen intern bereikbaar.

Impact

- **Meer aanvalskansen:** Een open databaseport op het internet trekt automatisch scans en loginpogingen aan.
- **Risico op onderschepping zonder TLS:** Als iemand op het netwerk mee kan kijken (of verkeer via een onbetrouwbaar segment loopt), kunnen credentials en data uitlekken.
- **Extra noise/last:** Meer logging en mogelijk extra load door geautomatiseerde aanvallen.

Reproduction

Bevestigen dat de database extern bereikbaar is

1. Scan de host: `nmap -sV -p 3306 <SERVER IP>`
2. Je ziet dat **port 3306** open staat als MariaDB/MySQL wordt herkend.
3. Als een dev run op de server zelf `ss -tulnp | grep 3306`.
4. Je ziet dat MariaDB op **0.0.0.0:3306** en/of **:::3306** luistert (via `docker-proxy`).

Bevestigen dat TLS niet afgedwongen wordt

1. Maak een verbinding zonder TLS: `mariadb --ssl=OFF -h <SERVER IP> -u <USER> -p`
2. In de database: `SHOW STATUS LIKE 'Ssl_version';`
3. Als `Ssl_version` leeg is, is er geen TLS in gebruik (dus onversleuteld verkeer).

Evidence

Nmap resultaat voor **voor port 3306**

```
nmap -sC -sV <SERVER_IP>
PORT STATE SERVICE VERSION
<SNIP>
3306/tcp open mysql MariaDB 12.1.2 |_ssl-date: TLS randomness does not represent time |
ssl-cert: Subject: commonName=MariaDB Server | Not valid before: 2025-12-19T21:53:45 |_Not
valid after: 2035-12-17T21:53:45
<SNIP>
```

Inloggen zonder TLS geeft ons de mogelijkheid om te checken of `Ssl_version` leeg is

```
za 17 jan - 01:28 ~
@christian mariadb --ssl=OFF -h 157.173.124.159 -u root -p -e "SHOW STATUS LIKE
'Ssl_version';"
Enter password:
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Ssl_version   |      |
+-----+-----+
```

Recommendation

Haal MariaDB van het internet af. Laat de database alleen intern bereikbaar zijn (bijv. alleen via Docker netwerk) of alleen vanaf de applicatiehost. **Forceer TLS voor DB-verbindingen.** Zorg dat clients niet zonder TLS kunnen verbinden en controleer dit met `Ssl_version/Ssl_cipher`. **Extra vangnet:** zet firewall rules neer zodat **port 3306** nooit per ongeluk weer publiek open komt te staan.

Retest

Status: Accepted

M 1: Non-Functional Rate Limiting	
Severity	Medium
OWASP	A06:2025 - Insecure Design
Affected components	• POST /api/auth/login • POST /api/auth/register
References	https://owasp.org/Top10/2025/A06_2025-Insecure_Design/

Overview

De applicatie beperkt het aantal inlog- en registratiepogingen niet (of niet effectief). Tijdens het testen konden we in korte tijd meerdere login- en register requests achter elkaar sturen zonder dat we werden vertraagd, geblokkeerd of een **429 Too Many Requests** terugkregen. Hierdoor kan een aanvaller dit endpoint makkelijk automatiseren voor brute-force/credential stuffing, of massaal accounts aanmaken.

Details

De API endpoints voor authenticatie lijken geen server-side throttling toe te passen. Hoewel rate limiting/throttling in de code of config aanwezig lijkt te zijn, wordt het in de praktijk niet afgedwongen: requests blijven gewoon verwerkt worden, en de API blijft normale **200/401** responses teruggeven.

Dit wijst meestal op één van deze oorzaken:

- Throttling is verkeerd geconfigureerd (bijv. verkeerde guards/middleware of global config)
- Throttling wordt niet toegepast op deze specifieke routes/controllers
- Throttling werkt alleen lokaal/dev en niet in productie
- Er is geen gedeelde store (bijv. Redis) waardoor limiting niet goed werkt bij meerdere instances.

Impact

Aanvallers kunnen geautomatiseerde inlogpogingen met hoge frequentie uitvoeren, waardoor de kans op accountcompromittering toeneemt. Daarnaast kunnen zij grote aantallen accounts aanmaken om resources te misbruiken of fraude mogelijk te maken.

Reproduction

1. Stuur binnen **60 seconden** ongeveer **10-15** inlogpogingen met foutieve credentials naar:
POST /api/auth/login
2. Observeer dat de API blijft antwoorden met normale responses (bijv. **401** of soms **200**) en **geen 429** terugstuurt, ook niet met vertraging.
3. Herhaal dit met registratie: stuur **10-15** registratiepogingen naar:
POST /api/auth/register\

4. Observeer dat er ook hier geen blokkade of throttling optreedt na de verwachte drempel.

Evidence

• Login Request 1

Request	Response
<pre>1 POST /api/auth/login HTTP/2 2 Host: 157.173.124.159 3 Cookie: refresh_token=to63vX2L-Y7vGcyhvr6CgRTVN6qtsE6FdkMLEL_cyaE 4 Content-Length: 58 5 Sec-Ch-Ua-Platform: "Linux" 6 Accept-Language: en-US,en;q=0.9 7 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24" 8 Content-Type: application/json 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: https://157.173.124.159 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://157.173.124.159/ 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 { "email": "test@email.com", "password": "MyTestPassword123!" }</pre>	<pre>1 HTTP/2 401 Unauthorized 2 Server: nginx/1.29.4 3 Date: Fri, 16 Jan 2026 11:03:37 GMT 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 137 6 X-Powered-By: Express 7 Access-Control-Allow-Origin: https://157.173.124.159 8 Vary: Origin 9 Access-Control-Allow-Credentials: true 10 X-Ratelimit-Limit: 5 11 X-Ratelimit-Remaining: 4 12 X-Ratelimit-Reset: 1 13 Retry-After: 900 14 Etag: W/"89-894WcLmYmTpBlfN009zlot8kCUY" 15 X-Frame-Options: SAMEORIGIN 16 X-Content-Type-Options: nosniff 17 Referrer-Policy: strict-origin-when-cross-origin 18 Strict-Transport-Security: max-age=86400 19 20 { "statusCode": 401, "message": "Invalid Credentials", "error": "Unauthorized", "path": "/api/auth/login", "timestamp": "2026-01-16T11:03:37.457Z" }</pre>

• Login Request 25

Request	Response
<pre>1 POST /api/auth/login HTTP/2 2 Host: 157.173.124.159 3 Cookie: refresh_token=to63vX2L-Y7vGcyhvr6CgRTVN6qtsE6FdkMLEL_cyaE 4 Content-Length: 58 5 Sec-Ch-Ua-Platform: "Linux" 6 Accept-Language: en-US,en;q=0.9 7 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24" 8 Content-Type: application/json 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: https://157.173.124.159 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://157.173.124.159/ 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 { "email": "test@email.com", "password": "MyTestPassword123!" }</pre>	<pre>1 HTTP/2 401 Unauthorized 2 Server: nginx/1.29.4 3 Date: Fri, 16 Jan 2026 11:03:37 GMT 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 137 6 X-Powered-By: Express 7 Access-Control-Allow-Origin: https://157.173.124.159 8 Vary: Origin 9 Access-Control-Allow-Credentials: true 10 X-Ratelimit-Limit: 5 11 X-Ratelimit-Remaining: 4 12 X-Ratelimit-Reset: 1 13 Retry-After: 900 14 Etag: W/"89-894WcLmYmTpBlfN009zlot8kCUY" 15 X-Frame-Options: SAMEORIGIN 16 X-Content-Type-Options: nosniff 17 Referrer-Policy: strict-origin-when-cross-origin 18 Strict-Transport-Security: max-age=86400 19 20 { "statusCode": 401, "message": "Invalid Credentials", "error": "Unauthorized", "path": "/api/auth/login", "timestamp": "2026-01-16T11:03:37.457Z" }</pre>

PoC voor het testen van rate limiting, verwachting is blokkade na 10 requests.

```
└─$ python3 rateLimit.py
Request 1: Status Code - 401, Time - 0.33 seconds
Request 2: Status Code - 401, Time - 0.32 seconds
Request 3: Status Code - 401, Time - 0.36 seconds
Request 4: Status Code - 401, Time - 0.33 seconds
Request 5: Status Code - 401, Time - 0.32 seconds
Request 6: Status Code - 401, Time - 0.36 seconds
Request 7: Status Code - 401, Time - 0.32 seconds
```

```
Request 8: Status Code - 401, Time - 0.33 seconds
Request 9: Status Code - 401, Time - 0.35 seconds
Request 10: Status Code - 401, Time - 0.35 seconds
Request 11: Status Code - 401, Time - 0.33 seconds
Request 12: Status Code - 401, Time - 0.33 seconds
Request 13: Status Code - 401, Time - 0.32 seconds
Request 14: Status Code - 401, Time - 0.36 seconds
Request 15: Status Code - 401, Time - 0.34 seconds
Request 16: Status Code - 401, Time - 0.32 seconds
Request 17: Status Code - 401, Time - 0.32 seconds
Request 18: Status Code - 401, Time - 0.32 seconds
Request 19: Status Code - 401, Time - 0.35 seconds
Request 20: Status Code - 401, Time - 0.32 seconds
Request 21: Status Code - 401, Time - 0.32 seconds
Request 22: Status Code - 401, Time - 0.36 seconds
Request 23: Status Code - 401, Time - 0.36 seconds
Request 24: Status Code - 401, Time - 0.32 seconds
Request 25: Status Code - 401, Time - 0.33 seconds
Request 26: Status Code - 401, Time - 0.35 seconds
Request 27: Status Code - 401, Time - 0.32 seconds
Request 28: Status Code - 401, Time - 0.36 seconds
Request 29: Status Code - 401, Time - 0.32 seconds
Request 30: Status Code - 401, Time - 0.32 seconds
Request 31: Status Code - 401, Time - 0.32 seconds
Request 32: Status Code - 401, Time - 0.32 seconds
Request 33: Status Code - 401, Time - 0.33 seconds
Request 34: Status Code - 401, Time - 0.33 seconds
Request 35: Status Code - 401, Time - 0.36 seconds
```

rateLimit.py

```
#!/usr/bin/env python3
import requests
requests.packages.urllib3.disable_warnings()
import time

refresh_token = "to63vX2L-Y7vGCyhvr6CgRTVN6qtsE8RDkMLEL_cyaE"
email = "test@email.com"
password = "MyTestPassword123!"
url = "https://157.173.124.159/api/auth/login"

headers = {
    "Cookie": f"refresh_token={refresh_token}",
    "Content-Type": "application/json"
}

data = {
    "email": email,
    "password": password
}

try:
    for i in range(35): # Send 20 requests
        start_time = time.time()
        response = requests.post(url, headers=headers, json=data, verify=False)
        end_time = time.time()
```

```
duration = end_time - start_time

print(f"Request {i+1}: Status Code - {response.status_code}, Time - {duration:.2f}
} seconds")
time.sleep(0.5) # Adjust to control the speed
except KeyboardInterrupt:
    print("\nStopped by user.")
```

Recommendation

- Zorg dat rate limiting/throttling in productie echt actief is op **/login** en **/register** (bijv. guard/middleware correct toepassen).
- Stuur na het bereiken van de limiet een duidelijke **429 Too Many Requests** terug, bij voorkeur met een cooldown.
- Gebruik een gedeelde throttling-store (bijv. **Redis**) zodat dit ook goed werkt in een multi-instance deployment.
- Overweeg aanvullende rate limiting op **WAF/CDN**-niveau als extra verdedigingslaag.

Retest

Status: Resolved

H 1: Interne applicatielogs worden gedeeld via Discord

Severity	High
OWASP	A02:2025 - Security Misconfiguration
Affected components	• UserIds • RefreshTokens
References	https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/ https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/

Overview

Een custom Discord-bot post applicatielogs in een Discord-kanaal. In deze logs staan **user IDs** en, belangrijker, **refresh tokens**. Refresh tokens zijn authenticatiegegevens, wie ze in handen krijgt kan (afhankelijk van de implementatie) nieuwe access tokens verkrijgen en sessies overnemen. Omdat Discord een extern platform is met gedeelde toegang en berichtgeschiedenis, vergroot dit het risico op account compromise aanzienlijk.

Details

De logging die naar Discord wordt gestuurd bevat gevoelige velden uit authenticatieflows. In de waargenomen logberichten komen onder andere:

- `userId`
- `refreshToken` (authenticatiecredential)

Deze gegevens horen niet in logs thuis, en zeker niet in logs die naar een chatplatform worden doorgestuurd. Discord-berichten zijn niet end-to-end encrypted en worden opgeslagen met historie, bij een gecompromitteerd Discord-account of verkeerde kanaalrechten kan deze informatie eenvoudig uitlekken.

Impact

- **Account takeover / sessie-overnemen:** Een gelekt refresh token kan mogelijk gebruikt worden om toegang te behouden en nieuwe sessies te verkrijgen.
- **Privilege impact:** Als een token van een admin/teacher account lekt, kan dit direct leiden tot misbruik van beheerfuncties of toegang tot andere gebruikersdata.
- **Grotere blast radius bij Discord-compromise:** Als een dev/admin Discord-account wordt overgenomen, krijgt de aanvaller direct "gratis" authenticatiegegevens uit de loggeschiedenis.
- **Privacy/recon:** user IDs maken correlatie en gerichte aanvallen makkelijker.

Reproduction

1. Open het Discord-kanaal waar de bot logmeldingen post (met een account dat toegang heeft).
2. Trigger een login/refresh-gerelateerde actie die een log event veroorzaakt.

3. Observeer dat de bot een logbericht post waarin `userId` en `refreshToken` zichtbaar zijn.
4. (Optioneel) Bevestig dat de token-string overeenkomt met het refresh token formaat dat de applicatie uitgeeft en niet gemaskeerd is.

Evidence

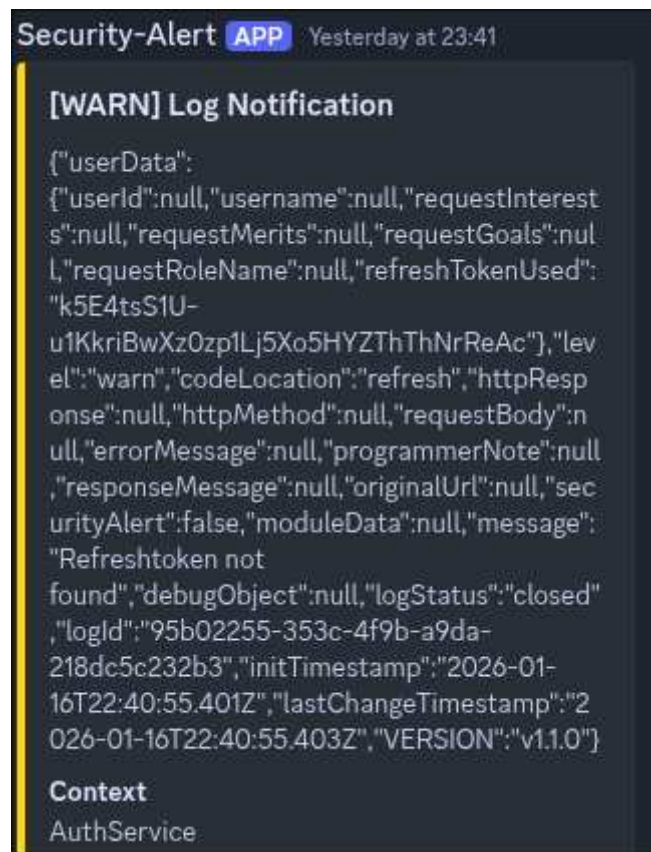
Voorbeeld UserId Output

```
[WARN] Log Notification

{"userData":{"userId":"bd298da7-4924-4b7b-9290-db5852cacd9f","username":null,"requestInterests":null,"requestMerits":null,"requestGoals":null,"requestRoleName":null,"refreshTokenUsed":null},"level":"warn","codeLocation":"/api/logs/archive","httpResponse":200,"httpMethod":"POST","requestBody":null,"errorMessage":null,"programmerNote":null,"responseMessage":null,"originalUrl":"/api/logs/archive","securityAlert":true,"moduleData":null,"message":"Logs archived successfully to archive-2026-01-16T15-07-06.log","debugObject":{"value":null,"fieldName":null,"filterData":null,"filterJson":null,"archiveFile":"archive-2026-01-16T15-07-06.log","FASTAPI_URL":null,"PAGE_SIZE":null,"response":null,"body":null},"logStatus":"closed","logId":"114fa6d6-25d3-4030-8f53-462c842d2ca5","initTimestamp":"2026-01-16T15:07:06.020Z","lastChangeTimestamp":"2026-01-16T15:07:06.021Z","VERSION":"v1.1.0"}

Context
LoggerController
```

Voorbeeld RefreshToken Output



Recommendation

- Use prepared statements throughout the application to effectively avoid SQL injection vulnerabilities. Prepared statements are parameterized statements and ensure that even if input values are manipulated, an attacker is unable to change the original intent of an SQL statement.
- Use existing stored procedures by default where possible. Typically, stored procedures are implemented as secure parameterized queries and thus protect against SQL injections.
- Always validate all user input. Ensure that only input that is expected and valid for the application is accepted. You should not sanitize potentially malicious input.
- To reduce the potential damage of a successful SQL Injection attack, you should minimize the assigned privileges of the database user used according to the principle of least privilege.
- For detailed information and assistance on how to prevent SQL Injection vulnerabilities, see OWASP's linked SQL Injection Prevention Cheat Sheet.

Retest

Status: Open

C 1: Exposed Sensitive Authentication Credentials	
Severity	Critical
OWASP	A04:2025 - Cryptographic Failures
Affected components	All SSH, Portainer, MariaDB credentials.
References	https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet

Overview

In het Discord-kanaal staan volledige inloggegevens voor kritieke systemen, waaronder **SSH**, **Portainer**, en **MariaDB**. De gedeelde accounts bevatten volledige privileges. Dit betekent dat iedereen met toegang tot dit kanaal (of iemand die een Discord-account van een developer/admin overneemt) direct kan inloggen op de server en beheerinterfaces. Dit is een directe route naar volledige systeemcompromittering.

Details

Tijdens de beoordeling is vastgesteld dat in Discord berichten aanwezig zijn **gebruikersnamen en wachtwoorden** voor:

- SSH toegang tot de server.
- Portainer beheerinterface
- MariaDB database (inclusief root-account en applicatieaccount)

Discord is een externe chatomgeving met berichtgeschiedenis en gedeelde toegang. Dit maakt credentials moeilijk te controleren (kopiëren, screenshots, forwards, ex-leden, role drift). Hierdoor is de vertrouwelijkheid van deze credentials niet te garanderen.

Impact

- **Directe account takeover / volledige server compromise:** Met SSH/Portainer credentials kan een aanvalleur systemen beheren, containers aanpassen, data exfiltreren en persistence inrichten.
- **Database compromise:** Met MariaDB root/app credentials kan data worden ingezien/aangepast/verwijderd.
- **Privilege escalatie:** Root-credentials verhogen de impact maximaal.
- **Langdurig risico:** Discord bewaart history, de exposure blijft bestaan totdat credentials worden geroteerd en berichten verwijderd.

Reproduction

1. Open het betreffende Discord-kanaal met een account dat toegang heeft.

2. Zoek de berichten waarin de credentials zijn gedeeld
3. Bevestig dat de berichten volledige credentials bevatten (username + password) voor SSH/Portainer/MariaDB.

Evidence

Credentials zitten in de discord #Links channel

Server SSH credentials

```
VPS:  
IP: 157.173.124.159  
  
(Root login niet mogelijk)  
User: root  
Password: Ouxu*****  
  
(sysadmin wel mogelijk)  
User: sysadmin  
Password: bCj%F*****
```

Portainer Credentials

```
Portainer login:  
  
URL: http://157.173.124.159:9000/#!/wizard  
User: admin  
Password: R1EG*****
```

MariaDB Database Login Credentials

```
MariaDB logins:  
  
IP: 157.173.124.159:3306  
DB naam: Keuzekompas_db  
  
User: appuser  
Password: %9*Wy*****  
  
User: root  
Password: W$uS*****
```

Recommendation

Directe Actie

- **Roteer alle gelekte credentials onmiddellijk** (SSH, Portainer, MariaDB). Behandel dit alsof ze al gecompromitteerd zijn.
- Verwijder de **Discord-berichten** met secrets en beperk kanaaltoegang tot strikt noodzakelijke rollen.
- **Controleer op misbruik:** Kijk naar SSH auth logs, Portainer audit/logins, DB Logs (waar mogelijk).

Structureel

- Gebruik een **secrets manager** of wachtwoordkluis (en deel nooit wachtwoorden via chat).
- Gebruik **least privilege**
 - Geen remote root login
 - DB app user alleen minimale rechten
 - Portainer accounts met MFA en beperkte rollen
- Voor SSH: bij voorkeur **key-based auth** en password auth uit (waar mogelijk).
- Introduceer "secret scanning", zodra een secret per ongeluk gedeeld is, directe rotatie.

Retest

Status: Open

5 Document History

Version	Date	Change
1.0	14-01-2026	Initial report
1.1	16-01-2026	Vulnerabilty Testing
1.2	17-01-2026	Methodology
1.3	18-01-2026	Afronding